

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization** and a **CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Code of Conduct:

I B.Jahnavi / 192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images. A **Convolutional Neural Network (CNN)** is more suitable than a traditional Fully Connected Neural Network (FCNN) because CNNs are specifically designed to process spatially structured data such as images.

1. Architectural Design

A traditional fully connected neural network connects **every neuron in one layer to every neuron in the next layer**. If an image is given as input, the image must usually be flattened into a one-dimensional vector.

For example, a grayscale image of size **224 × 224** contains:

pixels. If this is connected to just 1,000 neurons in the next layer, the number of weights is:

This produces a very large number of parameters.

A CNN, on the other hand, preserves the **height, width, and channel structure** of the image. It processes the image using convolutional filters and gradually learns important visual features.

2. Role of Convolutional Layers

A convolutional layer applies small filters or kernels over different regions of the image.

For example, a **3 × 3 filter** moves across the X-ray image and performs mathematical operations with local pixels.

Different filters can learn different features such as:

- Edges
- Lines
- Curves
- Textures
- Shapes
- Abnormal regions
- Lesion-like patterns

In medical images, early convolutional layers may identify basic structures, while deeper layers combine these features to identify more complex abnormalities.

3. Spatial Feature Extraction

One major advantage of CNNs is that they preserve **spatial relationships**.

For example, in an X-ray:

- The location of bones matters.
- The shape of organs matters.
- The relationship between different structures matters.
- The position and shape of abnormal regions can be important.

A fully connected network loses much of this spatial structure when the image is flattened.

CNNs use **local receptive fields**, meaning each neuron initially focuses on a small region of the image.

4. Pooling Layers

Pooling reduces the spatial dimensions of feature maps.

Common pooling methods include:

- Max pooling
- Average pooling

For example, a 2×2 max-pooling operation selects the largest value from each 2×2 region.

Pooling provides:

- Reduced feature-map size
- Lower computational cost
- Reduced number of parameters in later layers
- Some tolerance to small changes in object location
- Reduced sensitivity to noise

5. Number of Parameters

CNNs use **parameter sharing**. The same filter is reused across different parts of the image.

For example, a 3×3 filter has only:

weights, plus possibly a bias.

The same filter can detect an edge anywhere in the image.

In a fully connected network, different connections generally have independent weights, resulting in significantly more parameters.

6. Computational Complexity

Although CNNs perform many convolution operations, parameter sharing and local connectivity make them much more efficient for images than a fully connected architecture with comparable representational capability.

Fewer parameters also mean:

- Lower memory requirements
- Less storage
- More efficient training
- Reduced risk of overfitting

7. Overall Comparison

Feature	CNN	Fully Connected NN
Input	Image structure preserved	Usually flattened
Spatial information	Preserved	Largely lost
Feature extraction	Automatic	Less specialized
Parameters	Relatively fewer	Very large for images
Weight sharing	Yes	No
Image suitability	Excellent	Poorer

Feature	CNN	Fully Connected NN
Computational efficiency	High for images	Expensive for large images
Translation tolerance	Better	Poor
Overfitting risk	Lower with suitable design	Higher due to many parameters

Conclusion

CNNs are more suitable for X-ray classification because they automatically learn spatial and hierarchical features while using fewer parameters through **local connectivity and parameter sharing**. Convolutional layers extract features, pooling reduces dimensionality, and fully connected layers perform final classification. Therefore, CNNs are widely preferred for medical image classification tasks.

Q2. Different CNN Layers and Filters

A manufacturing company can use CNNs to detect defects such as cracks, scratches, holes, dents, discoloration, or irregular shapes in industrial products.

A CNN generally contains **convolutional layers, activation layers, pooling layers, and fully connected/classification layers**.

1. Convolutional Layer

The convolutional layer is primarily responsible for **feature extraction**.

A filter moves over the input image and produces a feature map.

For example:

Input image → Filter → Feature Map

Different filters detect different patterns.

A filter may learn to detect:

- Horizontal edges
- Vertical edges
- Diagonal edges

- Corners
- Textures
- Small cracks

The convolution operation reduces the need for manually designing features.

2. Activation Layer

After convolution, an activation function is generally applied.

The most common activation function in CNNs is **ReLU (Rectified Linear Unit)**:

It converts negative values to zero while retaining positive values.

The activation layer introduces **non-linearity**, allowing the CNN to learn complicated patterns.

Without nonlinear activation functions, stacking many linear operations would still produce a fundamentally linear transformation.

3. Pooling Layer

Pooling reduces the spatial size of feature maps.

For example, max pooling selects the maximum value within a region.

Its advantages include:

- Reducing computation
- Reducing memory requirements
- Reducing feature-map dimensions
- Providing some tolerance to small shifts
- Helping prevent overfitting

However, excessive pooling can remove useful defect information, especially for tiny cracks or scratches.

4. Fully Connected Layer

After several convolution and pooling operations, the extracted features are used for classification.

For example, the final layer might classify a product as:

- Normal
- Crack
- Scratch
- Dent
- Corrosion

Modern CNNs may use **global average pooling followed by a classification layer** instead of large fully connected layers.

5. Early-Layer Filters

Early convolutional layers usually learn simple features.

Examples include:

- Edges
- Lines
- Corners
- Basic textures
- Color/intensity transitions

For an industrial product, an early filter may detect a sharp boundary corresponding to the beginning of a crack.

6. Deeper-Layer Filters

Deeper layers combine simpler features to learn more complex patterns.

For example:

Pixels → Edges → Textures → Crack patterns → Defect type

A deeper layer may recognize the overall structure of a crack rather than just an individual edge.

7. Comparison

Layer	Main Function	Spatial Information	Computational Requirement
Convolution	Feature extraction	Preserves spatial structure	Moderate/High
Activation	Adds non-linearity	Preserves feature-map dimensions	Low
Pooling	Dimension reduction	Reduces spatial information	Low
Fully Connected	Classification	Spatial structure mostly converted to feature representation	Can be high

8. How They Work Together

A typical defect detection process is:

Input product image

↓

Convolution

Detect basic patterns.

↓

ReLU

Introduce non-linearity.

↓

Pooling

Reduce dimensions.



More convolution + ReLU

Detect increasingly complex defect patterns.



Classification layer

Predict the defect category.

Conclusion

The different CNN layers perform complementary tasks. Convolutional layers extract visual features, activation functions introduce non-linearity, pooling reduces spatial dimensions and computation, and classification layers determine the final defect category. Early filters detect simple patterns, while deeper filters detect complex defect structures. Therefore, the complete CNN architecture can automatically learn useful representations for industrial defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system may need to process thousands of high-resolution images. In this situation, CNN parameter sharing provides a major advantage over fully connected networks.

1. Parameter Sharing

In CNNs, the same convolutional filter is applied to different regions of the image.

For example, suppose a CNN has a 3×3 filter:

weights plus one bias.

These same weights can detect a particular feature throughout the entire image.

If an edge appears on the left side or right side, the same filter can detect it.

2. Independent Weights in Fully Connected Networks

In a fully connected network, every input neuron generally has a separate weight for each connection.

For a large high-resolution image, this can produce millions of parameters even in the first layer.

This leads to:

- High memory usage
- Greater computation
- Longer training
- Greater risk of overfitting

3. Memory Requirement

CNN parameter sharing dramatically reduces the number of parameters.

For example, a convolution layer with 64 filters of size 3×3 and 3 input channels requires: weights, excluding biases.

A fully connected layer connecting a large image directly to many neurons could require millions of independent weights.

4. Computational Complexity

Parameter sharing makes CNNs computationally efficient because the network does not need to learn a separate detector for every possible image location.

Instead, it learns a small number of useful filters and reuses them.

This is especially valuable when processing:

- Surveillance videos
- High-resolution images
- Large datasets
- Real-time camera feeds

5. Feature Detection

Parameter sharing allows a filter to detect the same feature in different locations.

For example, a filter trained to recognize an edge can identify an edge:

- At the top
- At the bottom
- On the left
- On the right

of the image.

This is called **translation equivariance** of convolution: when the input shifts, the corresponding feature map also shifts.

Pooling and other architectural mechanisms can provide some degree of **translation tolerance/invariance**, although CNNs are not perfectly invariant to all translations.

6. Learning Efficiency

Suppose the surveillance system contains thousands of images.

A CNN does not need to learn separate parameters for every location of a feature. The same filter can learn from many examples across many locations.

This means that the model can learn general visual patterns more efficiently.

7. Comparison

Feature	CNN Parameter Sharing	Fully Connected Independent Weights
Weight usage	Same filter reused	Separate weights
Number of parameters	Low	Very high for images
Memory	Lower	Higher
Training	More efficient	More expensive
Spatial information	Preserved	Largely lost after flattening
Feature detection	Location-flexible	Location-specific

Feature	CNN	Parameter Sharing	Fully Connected	Independent Weights
---------	-----	-------------------	-----------------	---------------------

Overfitting	Generally lower		Higher	
Large image processing	Suitable		Expensive	

Conclusion

Parameter sharing is one of the most important reasons CNNs are effective for large-scale image processing. The same convolutional filters can detect features at different locations, reducing the number of parameters and memory requirements. For surveillance systems

Q4. AlexNet vs ResNet

AlexNet and ResNet are important CNN architectures, but they were designed for different generations of deep learning.

1. Network Depth

AlexNet is a relatively shallow CNN compared with modern architectures. It contains several convolutional and fully connected layers.

ResNet can have much greater depth, such as:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

The larger depth allows ResNet to learn increasingly complex hierarchical features.

2. Layer Organization

AlexNet uses a straightforward sequence:

Convolution → Activation → Pooling → Fully Connected

ResNet introduces **residual blocks**.

A residual block allows the input to bypass one or more layers through a shortcut connection.

Conceptually:

where:

- = original input
- = transformation learned by the convolutional layers
- = residual output

3. Vanishing Gradient Problem

As networks become deeper, gradients may become very small during backpropagation. This is known as the **vanishing-gradient problem**.

It makes very deep networks difficult to train.

ResNet addresses this using **skip/shortcut connections**, which provide an easier path for gradient propagation.

Therefore, ResNet can successfully train much deeper networks.

4. Feature Extraction

AlexNet learns features from:

- Edges
- Textures
- Shapes
- Object patterns

ResNet also performs hierarchical feature extraction but can learn significantly more complex representations due to its greater depth.

5. Parameter Requirements

AlexNet has a relatively large number of parameters, especially because of its fully connected layers.

ResNet's parameter count depends on the variant. Interestingly, deeper ResNet versions can be parameter-efficient because architectures such as ResNet-50 use bottleneck blocks.

Therefore, **depth alone does not determine parameter count**.

6. Training Difficulty

AlexNet is easier to understand and train because it is comparatively simple.

A very deep conventional network can be difficult to optimize.

ResNet makes deep training easier through residual connections.

7. Computational Cost

AlexNet generally requires less computation than very deep ResNet variants.

ResNet-50, ResNet-101, and ResNet-152 require more computation than AlexNet, but they generally provide much stronger feature representations.

The exact cost depends on:

- Input resolution
- ResNet version
- Batch size
- Hardware
- Number of operations

8. Comparison

Feature	AlexNet	ResNet
Depth	Relatively shallow	Very deep
Main innovation	Deep CNN with ReLU, pooling and large-scale image classification	Residual/skip connections
Feature learning	Good	More powerful
Skip connections	No	Yes
Very deep training	Difficult	Much easier
Vanishing gradient	More problematic	Significantly reduced
Parameters	Large, especially FC layers	Depends on variant

Feature	AlexNet	ResNet
Computational cost	Lower than deep ResNets	Higher for deeper variants
Complex recognition	Good	Excellent
Modern applications	Limited compared with newer architectures	Widely used

9. Recommendation

For a complex image recognition application, **ResNet is generally the better choice.**

The main reasons are:

1. It supports very deep networks.
2. Residual connections improve gradient propagation.
3. It learns complex hierarchical features.
4. It generally provides stronger image recognition performance.
5. It is more scalable for complex computer vision tasks.

AlexNet is historically important and useful for understanding the development of CNNs, but ResNet is generally more appropriate for demanding modern image recognition applications.

Q5. Regularized CNN vs Non-Regularized CNN

The agricultural organization observes very high training accuracy but poor performance on unseen leaf images. This is a classic indication of **overfitting**.

Overfitting occurs when the CNN learns the training images too specifically instead of learning general patterns associated with plant diseases.

For example, the model might memorize:

- Background patterns
- Lighting conditions
- Specific leaf positions
- Image noise
- Individual training examples

instead of learning the actual disease characteristics.

1. CNN Without Regularization

A non-regularized CNN may have:

- Very high training accuracy
- Low training loss
- Poor validation accuracy
- Poor test accuracy

This means the model performs well on known examples but fails to generalize.

For example:

Training accuracy = 98%

Testing accuracy = 70%

This large difference indicates potential overfitting.

2. Dropout

Dropout randomly disables a proportion of neurons during training.

For example, with a dropout rate of 0.5, approximately half of the selected activations are randomly dropped during a training step.

This prevents the network from depending too heavily on particular neurons.

Benefits:

- Reduces overfitting
- Encourages distributed feature learning
- Improves generalization

However, too much dropout can make learning slower or cause underfitting.

3. Data Augmentation

Data augmentation creates modified versions of existing training images.

For plant leaves, suitable transformations may include:

- Rotation
- Horizontal flipping
- Small translations
- Zooming
- Cropping
- Brightness variation
- Contrast variation

For example, one leaf image can produce several realistic training variations.

This increases the effective diversity of the dataset.

Why is this important?

Real agricultural images may be captured under:

- Different sunlight
- Different camera angles
- Different distances

- Different backgrounds
- Different leaf orientations

Data augmentation helps the model become more robust to these variations.

4. Batch Normalization

Batch normalization normalizes intermediate activations during training.

It can:

- Improve training stability
- Allow more stable optimization
- Often speed up training
- Help the network train effectively with deeper architectures

Batch normalization is primarily an optimization technique rather than a traditional regularizer, although it can also have a regularizing effect.

5. Comparison

Feature	Without Regularization	With Regularization
Training accuracy	Often very high	Usually high
Test accuracy	Often lower	Usually improved
Overfitting	High risk	Reduced
Generalization	Poorer	Better
Dropout	Not used	Reduces neuron dependency
Data augmentation	Not used	Increases data diversity
Batch normalization	Not used	Improves training stability
Real-world reliability	Lower	Higher

6. Which Techniques Should Be Applied?

For plant disease classification, a **combination** of techniques is preferable.

Data Augmentation

This should be strongly considered because agricultural images naturally vary in orientation, lighting, scale, and background.

Dropout

Dropout can be applied, particularly around the classification portion of the network, when the model is overfitting.

Batch Normalization

Batch normalization can be included in convolutional blocks to improve training stability and optimization.

Additional Measures

Other useful techniques include:

- Early stopping
- Weight decay/L2 regularization
- Transfer learning
- Cross-validation where appropriate
- Monitoring validation loss
- Collecting more diverse images

7. Recommended Approach

A practical architecture could follow:

Leaf Image

↓

Data Augmentation

↓

Convolution

↓

Batch Normalization

↓

ReLU

↓

Pooling

↓

More Convolutional Blocks

↓

Dropout

↓

Classification Layer

↓

Disease Prediction

The training process should monitor **validation performance**, not only training accuracy.

For example:

Model	Training Accuracy	Validation Accuracy	Interpretation
No regularization	98%	69%	Strong overfitting
With augmentation	94%	82%	Better generalization
Augmentation + dropout	92%	86%	Reduced overfitting
Augmentation + dropout + BN	93%	88%	More stable and generalized

These values are illustrative, not experimental results.

Conclusion

The regularized CNN is more appropriate for real-world plant disease classification. Data augmentation increases the diversity of training examples, dropout reduces dependence on specific neurons, and batch normalization improves training stability. These techniques together can reduce overfitting and improve generalization to unseen leaf images.

Overall recommendation: For a real agricultural deployment, use **data augmentation + suitable batch normalization + moderate dropout**, while monitoring validation performance and using early stopping when necessary. This provides a better balance between training accuracy, generalization, computational cost, and reliability.